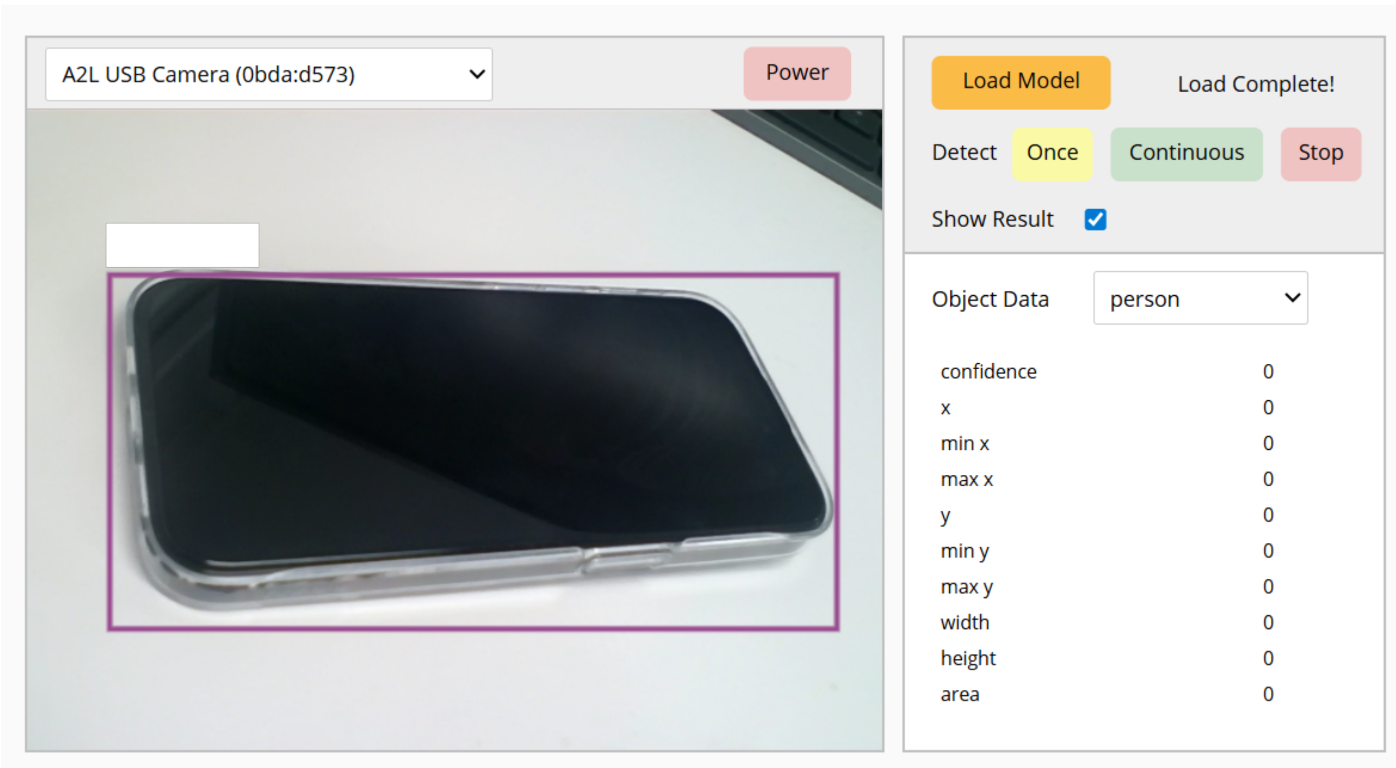
Open Dashboard

Open Dashboard is not a block that can be used in block coding,
but it allows you to open a dashboard where you can check how the model used in the extension module is applied.

Dashboard Screen

When you click Open Dashboard, you can see the following screen.



Detailed Features

Power

Turns the selected camera on or off.

Load Model

Loads the trained object detection model.
This is a required step to use the “Object Detection” extension module.

Detect

Starts or stops object detection.
You can choose whether to run it only once with the Once button, or continuously with the Continuous button.
You can also stop detection using the Stop button.

Show Result

Displays object detection results on the camera screen.

Object Data

Displays data values according to the selected object.

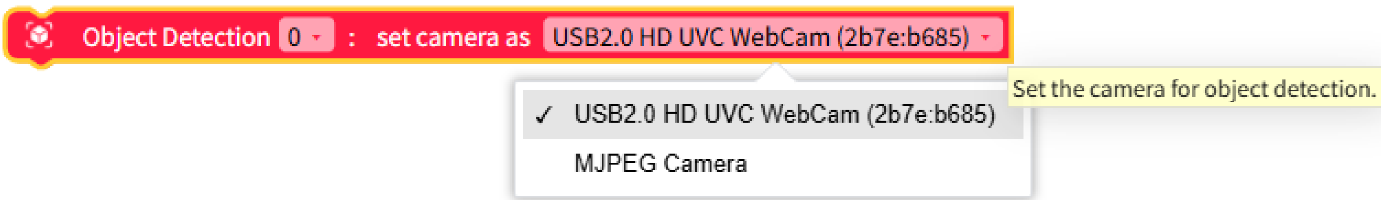
Dropdown Options and Input Values

Name	Type	Description	Range / Type
object	Dropdown Option	Object	person, bicycle, car, motorcycle, airplane, bus, train, truck, boat, traffic light, fire hydrant, stop sign, parking meter, bench, bird, cat, dog, horse, sheep, cow, elephant, bear, zebra, giraffe, backpack, umbrella, handbag, tie, suitcase, frisbee, skis, snowboard, sports ball, kite, baseball bat, baseball glove, skateboard, surfboard, tennis racket, bottle, wine glass, cup, fork, knife, spoon, bowl, banana, apple, sandwich, orange, broccoli, carrot, hot dog, pizza, donut, cake, chair, couch, potted plant, bed, dining table, toilet, tv, laptop, mouse, remote, keyboard, cell phone, microwave, oven, toaster, sink, refrigerator, book, clock, vase, scissors, teddy bear, hair drier, toothbrush

Blocks

Select Camera

Selects the camera to use for the Object Detection module.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
camera	Dropdown Option	Camera to use	Connected camera list

JavaScript Code

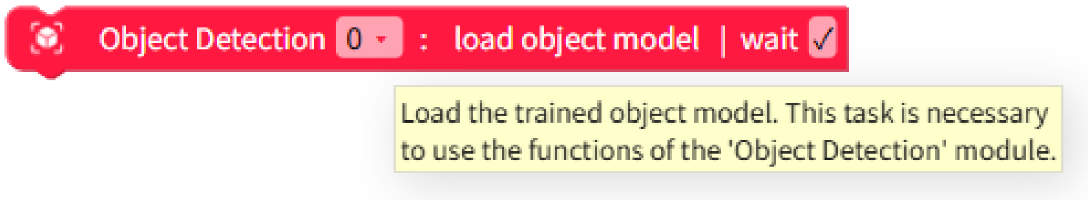
```
// Set a specific camera for object detection (id is an example)
$('ObjectDetection*0:camera.deviceId').d =
'035658da47183882a695a82c45b8f3e9ae50cef47945ccdc3f31e1ae1fbca9cb';
```

Python Code

```
# Set a specific camera for object detection (id is an example)
__('ObjectDetection*0:camera.deviceId').d =
'035658da47183882a695a82c45b8f3e9ae50cef47945ccdc3f31e1ae1fbca9cb'
```

Load Object Model

Loads the trained object model. This step is required to use the features of the "Object Detection" module.
If wait is checked, it waits until model loading is completed.
However, if wait is checked, it can only be used inside an async function.



JavaScript Code

```
// Load object model | Wait O
$('ObjectDetection*0:load_model').d = 1;
await $('ObjectDetection*0:!load_model').w();

// Load object model | Wait X
$('ObjectDetection*0:load_model').d = 1;
```

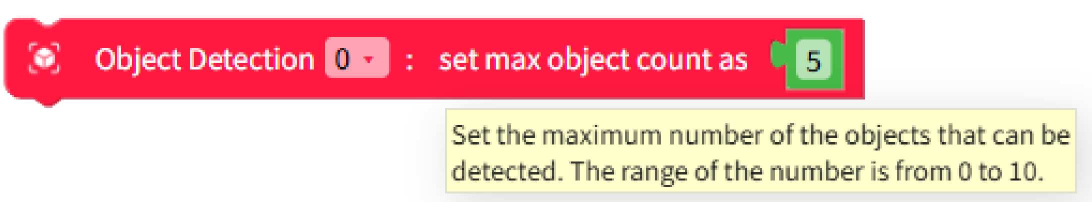
Python Code

```
# Load object model | Wait O
__('ObjectDetection*0:load_model').d = 1
await __('ObjectDetection*0:!load_model').w()

# Load object model | Wait X
__('ObjectDetection*0:load_model').d = 1
```

Set Max Number of Objects to Detect

Sets the maximum number of objects that can be detected.

The range is 0 ~ 10.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
pivot	Input	Number of objects to detect	Integer 0 ~ 10

JavaScript Code

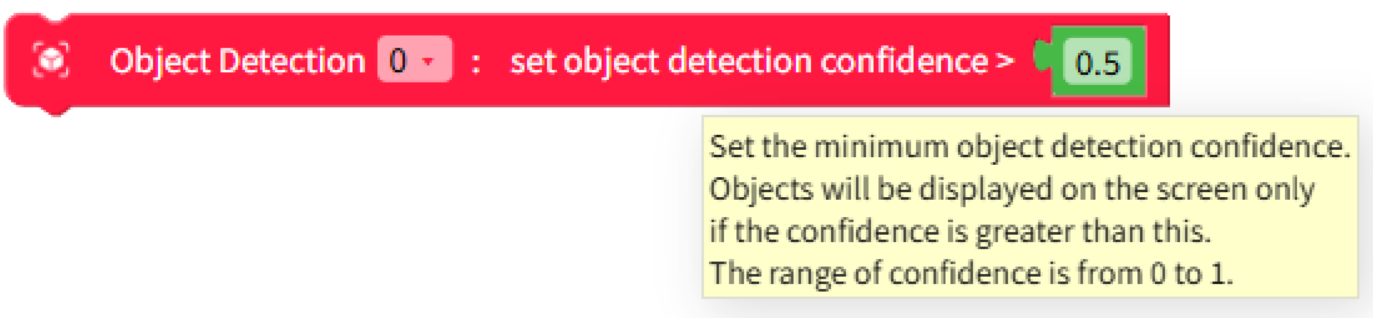
```
// Set the max number of objects to 5
$('ObjectDetection*0:max_count').d = 5;
```

Python Code

```
# Set the max number of objects to 5
__('ObjectDetection*0:max_count').d = 5
```

Set Minimum Detection Confidence

Sets the minimum confidence threshold for object detection.
Only results with confidence greater than or equal to this value are displayed.
The confidence range is 0 ~ 1.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
reliability	Input	Object detection confidence	Real number 0 ~ 1

JavaScript Code

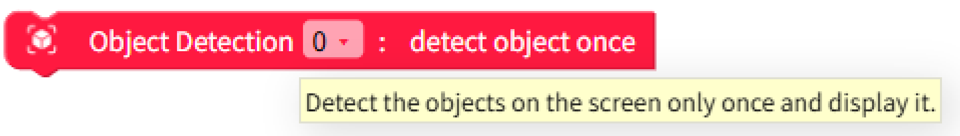
```
// Set minimum confidence threshold to 0.5
$('ObjectDetection*0:confidence').d = 0.5;
```

Python Code

```
# Set minimum confidence threshold to 0.5
__('ObjectDetection*0:confidence').d = 0.5
```

Detect Once

Detects objects currently on the screen and displays the result only once.



JavaScript Code

```
// Detect objects once
$('ObjectDetection*0:detect.once').d = 1;
```

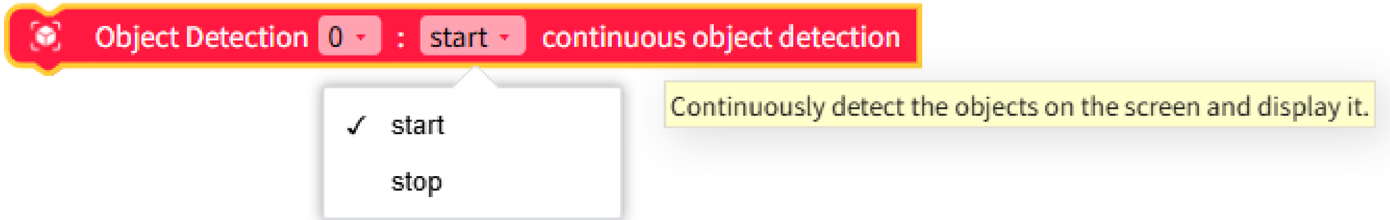
Python Code

```
# Detect objects once
__('ObjectDetection*0:detect.once').d = 1
```

Detect Continuously

Starts or stops continuous object detection.

When continuous detection starts, it keeps tracking and displaying objects on the screen.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
toggle	Dropdown Option	Object detection	Start (1), Stop (0)

JavaScript Code

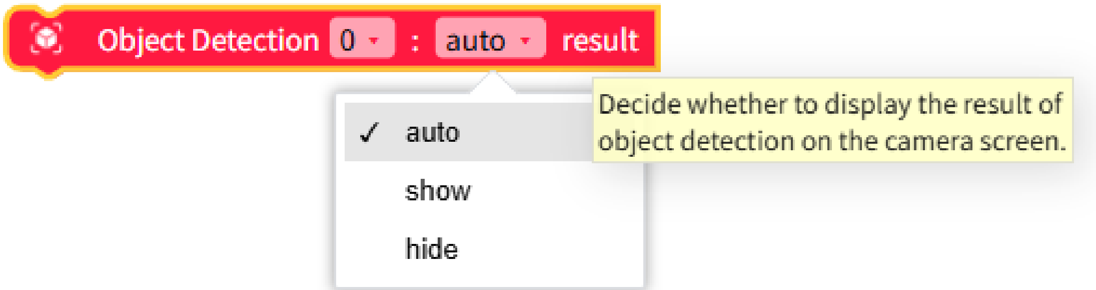
```
// Start continuous detection
$('ObjectDetection*0:detect.continuous').d = 1;

// Stop continuous detection
$('ObjectDetection*0:detect.continuous').d = 0;
```

Python Code

```
# Start continuous detection
__('ObjectDetection*0:detect.continuous').d = 1

# Stop continuous detection
__('ObjectDetection*0:detect.continuous').d = 0
```

Show Object Detection Result

Decides whether to display object detection results on the camera screen.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
toggle	Dropdown Option	Show results	Show (1), Hide (0)

JavaScript Code

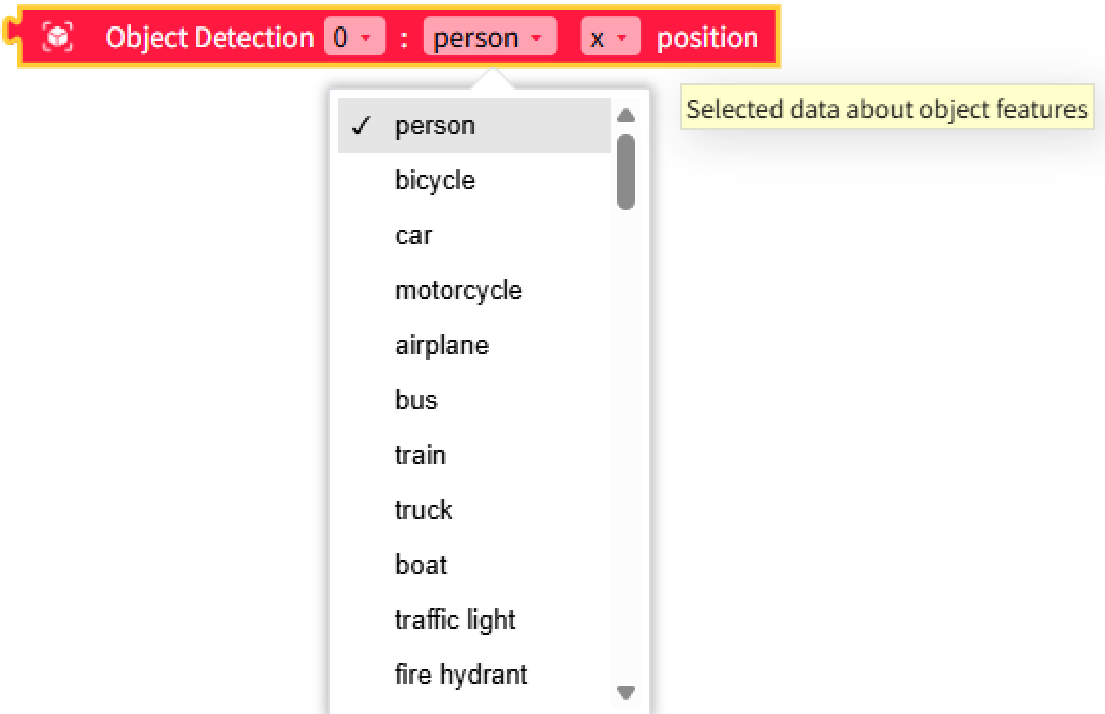
```
// Show detection results
$('ObjectDetection*0:display').d = 1;

// Hide detection results
$('ObjectDetection*0:display').d = 0;
```

Python Code

```
# Show detection results
__('ObjectDetection*0:display').d = 1

# Hide detection results
__('ObjectDetection*0:display').d = 0
```

Object Data

Returns data for the selected object.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
Name	Type	Description	Range / Type
object	Dropdown Option	Object	Person (0), Bicycle (1), Car (2), Motorcycle (3), Airplane (4), Bus (5), Train (6), Truck (7), Boat (8), Traffic light (9), Fire hydrant (10), Stop sign (11), Parking meter (12), Bench (13), Bird (14), Cat (15), Dog (16), Horse (17), Sheep (18), Cow (19), Elephant (20), Bear (21), Zebra (22), Giraffe (23), Backpack (24), Umbrella (25), Handbag (26), Tie (27), Suitcase (28), Frisbee (29), Skis (30), Snowboard (31), Sports ball (32), Kite (33), Baseball bat (34), Baseball glove (35), Skateboard (36), Surfboard (37), Tennis racket (38), Bottle (39), Wine glass (40), Cup (41), Fork (42), Knife (43), Spoon (44), Bowl (45), Banana (46), Apple (47), Sandwich (48), Orange (49), Broccoli (50), Carrot (51), Hot dog (52), Pizza (53), Donut (54), Cake (55), Chair (56), Couch (57),

Name	Type	Description	Range / Type
axis	Dropdown Option	Coordinate axis	x, y

JavaScript Code

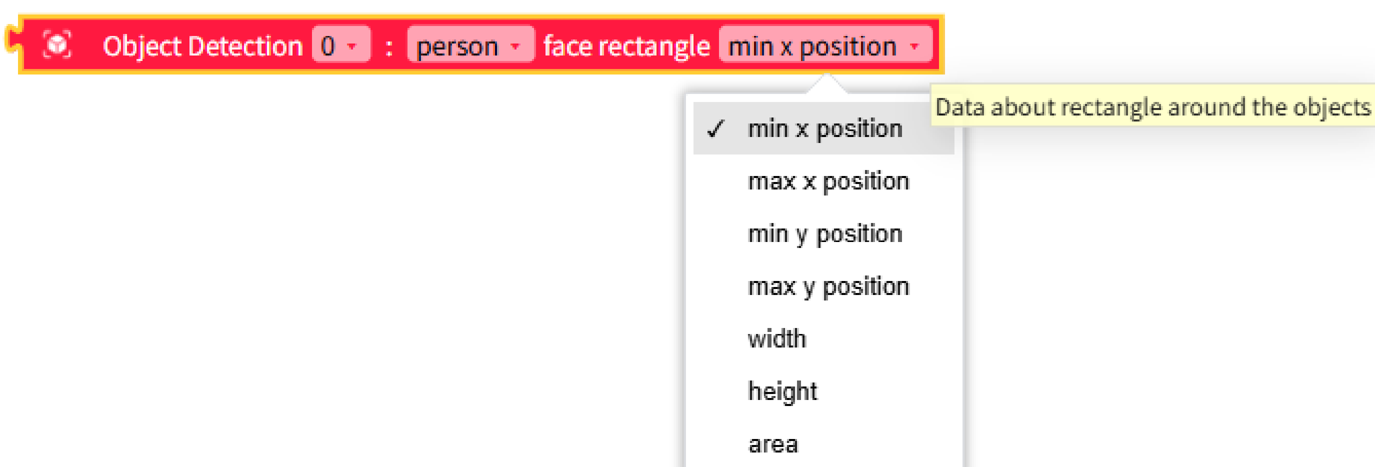
```
// Person (0) x coordinate
$('ObjectDetection*0:object.x').d[0];

// Toothbrush (79) y coordinate
$('ObjectDetection*0:object.y').d[79];
```

Python Code

```
# Person (0) x coordinate
__('ObjectDetection*0:object.x').d[0]

# Toothbrush (79) y coordinate
__('ObjectDetection*0:object.y').d[79]
```

Bounding Box Data Around Object

Defines the detected object area as a rectangle and returns rectangle data.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
Name	Type	Description	Range / Type
object	Dropdown Option	Object	Person (0), Bicycle (1), Car (2), Motorcycle (3), Airplane (4), Bus (5), Train (6), Truck (7), Boat (8), Traffic light (9), Fire hydrant (10), Stop sign (11), Parking meter (12), Bench (13), Bird (14), Cat (15), Dog (16), Horse (17), Sheep (18), Cow (19), Elephant (20), Bear (21), Zebra (22), Giraffe (23), Backpack (24), Umbrella (25), Handbag (26), Tie (27), Suitcase (28), Frisbee (29), Skis (30), Snowboard (31), Sports ball (32), Kite (33), Baseball bat (34), Baseball glove (35), Skateboard (36), Surfboard (37), Tennis racket (38), Bottle (39), Wine glass (40), Cup (41), Fork (42), Knife (43), Spoon (44), Bowl (45), Banana (46), Apple (47), Sandwich (48), Orange (49), Broccoli (50), Carrot (51), Hot dog (52), Pizza (53), Donut (54), Cake (55), Chair (56), Couch (57),

Name	Type	Description	Range / Type
axis	Dropdown Option	Rectangle data	min_x, max_x, min_y, max_y, width, height, area

JavaScript Code

```
// Bus (5) rectangle min x
$('ObjectDetection*0:object.face.min_x').d[5];

// Bus (5) rectangle max x
$('ObjectDetection*0:object.face.max_x').d[5];

// Bus (5) rectangle min y
$('ObjectDetection*0:object.face.min_y').d[5];

// Bus (5) rectangle max y
$('ObjectDetection*0:object.face.max_y').d[5];

// Bus (5) rectangle width
$('ObjectDetection*0:object.face.width').d[5];

// Bus (5) rectangle height
$('ObjectDetection*0:object.face.height').d[5];

// Bus (5) rectangle area
$('ObjectDetection*0:object.face.area').d[5];
```

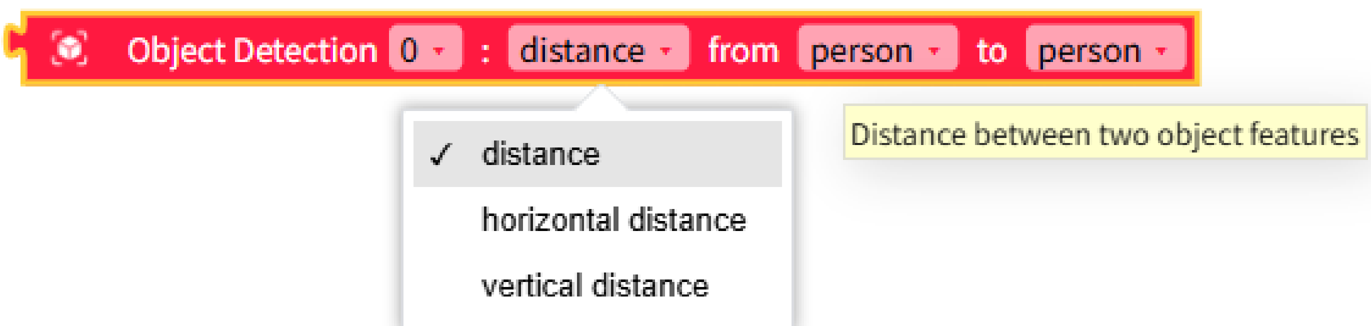
Python Code

```
# Bus (5) rectangle min x
__('ObjectDetection*0:object.face.min_x').d[5]
```

```
# Bus (5) rectangle max x
__('ObjectDetection*0:object.face.max_x').d[5]

# Bus (5) rectangle min y
__('ObjectDetection*0:object.face.min_y').d[5]

# Bus (5) rectangle max y
__('ObjectDetection*0:object.face.max_y').d[5]

# Bus (5) rectangle width
__('ObjectDetection*0:object.face.width').d[5]

# Bus (5) rectangle height
__('ObjectDetection*0:object.face.height').d[5]

# Bus (5) rectangle area
__('ObjectDetection*0:object.face.area').d[5]
```

Distance Between Two Objects

Returns the distance between two selected objects.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
Name	Type	Description	Range / Type
object 1	Dropdown Option	Object	Person (0), Bicycle (1), Car (2), Motorcycle (3), Airplane (4), Bus (5), Train (6), Truck (7), Boat (8), Traffic light (9), Fire hydrant (10), Stop sign (11), Parking meter (12), Bench (13), Bird (14), Cat (15), Dog (16), Horse (17), Sheep (18), Cow (19), Elephant (20), Bear (21), Zebra (22), Giraffe (23), Backpack (24), Umbrella (25), Handbag (26), Tie (27), Suitcase (28), Frisbee (29), Skis (30), Snowboard (31), Sports ball (32), Kite (33), Baseball bat (34), Baseball glove (35), Skateboard (36), Surfboard (37), Tennis racket (38), Bottle (39), Wine glass (40), Cup (41), Fork (42), Knife (43), Spoon (44), Bowl (45), Banana (46), Apple (47), Sandwich (48), Orange (49), Broccoli (50), Carrot (51), Hot dog (52), Pizza (53), Donut (54), Cake (55), Chair (56), Couch (57),

Name	Type	Description	Range / Type
object 2	Dropdown Option	Object	Person (0), Bicycle (1), Car (2), Motorcycle (3), Airplane (4), Bus (5), Train (6), Truck (7), Boat (8), Traffic light (9), Fire hydrant (10), Stop sign (11), Parking meter (12), Bench (13), Bird (14), Cat (15), Dog (16), Horse (17), Sheep (18), Cow (19), Elephant (20), Bear (21), Zebra (22), Giraffe (23), Backpack (24), Umbrella (25), Handbag (26), Tie (27), Suitcase (28), Frisbee (29), Skis (30), Snowboard (31), Sports ball (32), Kite (33), Baseball bat (34), Baseball glove (35), Skateboard (36), Surfboard (37), Tennis racket (38), Bottle (39), Wine glass (40), Cup (41), Fork (42), Knife (43), Spoon (44), Bowl (45), Banana (46), Apple (47), Sandwich (48), Orange (49), Broccoli (50), Carrot (51), Hot dog (52), Pizza (53), Donut (54), Cake (55), Chair (56), Couch (57), Potted plant (58), Bed

Name	Type	Description	Range / Type
distance	Dropdown Option	Distance type	distance, horizontal distance, vertical distance

JavaScript Code

```
// Distance from person (0) to person (0)
Math.sqrt( Math.pow(($('ObjectDetection*0:object.x').d[0] -
$('ObjectDetection*0:object.x').d[0]), 2) +
Math.pow(($('ObjectDetection*0:object.y').d[0] -
$('ObjectDetection*0:object.y').d[0]), 2) );

// Horizontal distance from person (0) to bicycle (1)
Math.abs($('ObjectDetection*0:object.x').d[1] -
$('ObjectDetection*0:object.x').d[0]);

// Vertical distance from car (2) to bicycle (1)
Math.abs($('ObjectDetection*0:object.y').d[1] -
$('ObjectDetection*0:object.y').d[2]);
```

Python Code

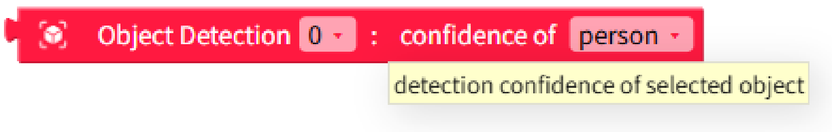
```
# Distance from person (0) to person (0)
math.sqrt( math.pow((__('ObjectDetection*0:object.x').d[0] -
__('ObjectDetection*0:object.x').d[0]), 2) +
math.pow((__('ObjectDetection*0:object.y').d[0] -
__('ObjectDetection*0:object.y').d[0]), 2) )

# Horizontal distance from person (0) to bicycle (1)
math.fabs(__('ObjectDetection*0:object.x').d[1] -
__('ObjectDetection*0:object.x').d[0])
```

```
# Vertical distance from car (2) to bicycle (1)
math.fabs(__('ObjectDetection*0:object.y').d[1] -
__('ObjectDetection*0:object.y').d[2])
```

Object Confidence (Reliability)

Returns the confidence (reliability) that the selected object is correct.

The returned value is a real number between 0 and 1.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
Name	Type	Description	Range / Type
object	Dropdown Option	Object	Person (0), Bicycle (1), Car (2), Motorcycle (3), Airplane (4), Bus (5), Train (6), Truck (7), Boat (8), Traffic light (9), Fire hydrant (10), Stop sign (11), Parking meter (12), Bench (13), Bird (14), Cat (15), Dog (16), Horse (17), Sheep (18), Cow (19), Elephant (20), Bear (21), Zebra (22), Giraffe (23), Backpack (24), Umbrella (25), Handbag (26), Tie (27), Suitcase (28), Frisbee (29), Skis (30), Snowboard (31), Sports ball (32), Kite (33), Baseball bat (34), Baseball glove (35), Skateboard (36), Surfboard (37), Tennis racket (38), Bottle (39), Wine glass (40), Cup (41), Fork (42), Knife (43), Spoon (44), Bowl (45), Banana (46), Apple (47), Sandwich (48), Orange (49), Broccoli (50), Carrot (51), Hot dog (52), Pizza (53), Donut (54), Cake (55), Chair (56), Couch (57),

JavaScript Code

```
// Confidence that the detected object is a person (0)
$('ObjectDetection*0:object.confidence').d[0];

// Confidence that the detected object is a bicycle (1)
$('ObjectDetection*0:object.confidence').d[1];
```

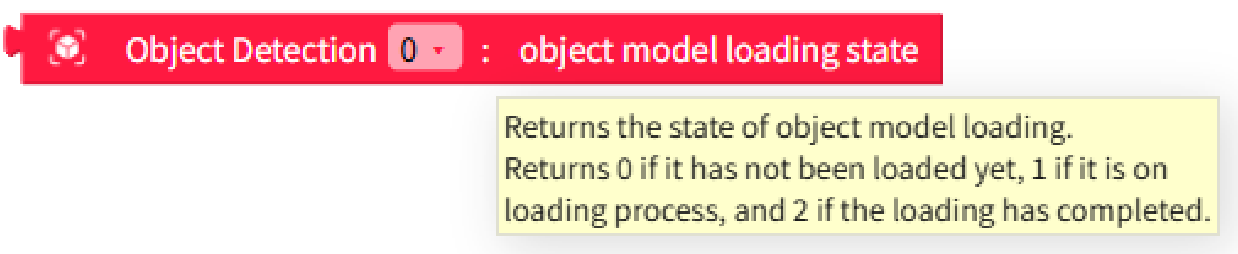
Python Code

```
# Confidence that the detected object is a person (0)
__('ObjectDetection*0:object.confidence').d[0]

# Confidence that the detected object is a bicycle (1)
__('ObjectDetection*0:object.confidence').d[1]
```

Model Loading State

Returns the model loading state.

Returns 0 if not loaded yet, 1 if loading, and 2 if loading is complete.



JavaScript Code

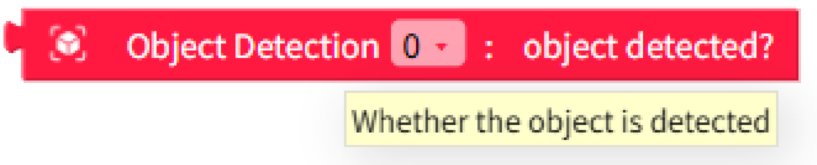
```
// Model loading state value
$('ObjectDetection*0:model_state').d;
```

Python Code

```
# Model loading state value
__('ObjectDetection*0:model_state').d
```

Was Any Object Detected?

Returns whether any object was detected as **True (1) / False (0)**.



JavaScript Code

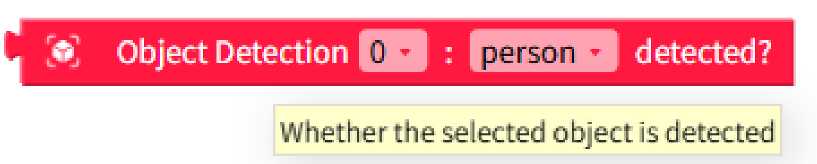
```
// Was any object detected?
$('ObjectDetection*0:detected').d;
```

Python Code

```
# Was any object detected?
__('ObjectDetection*0:detected').d
```

Was ~ Detected?

Returns whether the selected object was detected as **True (1) / False (0)**.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
Name	Type	Description	Range / Type
object	Dropdown Option	Object	Person (0), Bicycle (1), Car (2), Motorcycle (3), Airplane (4), Bus (5), Train (6), Truck (7), Boat (8), Traffic light (9), Fire hydrant (10), Stop sign (11), Parking meter (12), Bench (13), Bird (14), Cat (15), Dog (16), Horse (17), Sheep (18), Cow (19), Elephant (20), Bear (21), Zebra (22), Giraffe (23), Backpack (24), Umbrella (25), Handbag (26), Tie (27), Suitcase (28), Frisbee (29), Skis (30), Snowboard (31), Sports ball (32), Kite (33), Baseball bat (34), Baseball glove (35), Skateboard (36), Surfboard (37), Tennis racket (38), Bottle (39), Wine glass (40), Cup (41), Fork (42), Knife (43), Spoon (44), Bowl (45), Banana (46), Apple (47), Sandwich (48), Orange (49), Broccoli (50), Carrot (51), Hot dog (52), Pizza (53), Donut (54), Cake (55), Chair (56), Couch (57),

JavaScript Code

```
// Was person (0) detected?
$('ObjectDetection*0:object.confidence').d[0] >=
$('ObjectDetection*0:confidence');
```

Python Code

‘‘‘python # Was person (0) detected? __(’ObjectDetection*0:object.confidence’).d[0] >= __(’ObjectDetection*0:confidence’).d